

CSS Grid Tracks Alignment Properties

Complete Guide: Easy English Explanation

CSS Grid Mastery

May 10, 2026

Contents

1	Introduction	2
1.1	What is CSS Grid Tracks Alignment?	2
1.2	Tracks vs Items: Understanding the Difference	2
1.3	Why Use Track Alignment?	2
2	Core Track Alignment Properties	3
2.1	justify-content: Aligning Columns Horizontally	3
2.1.1	What Does It Do?	3
2.1.2	Available Values	3
2.1.3	Basic Example	3
2.1.4	Real Example: space-between	4
2.2	align-content: Aligning Rows Vertically	4
2.2.1	What Does It Do?	4
2.2.2	Available Values	4
2.2.3	Basic Example	4
2.2.4	Real Example: space-around	5
2.3	place-content: Combine Both Alignments	5
2.3.1	What Does It Do?	5
2.3.2	Syntax	5
2.3.3	Example	5
3	Understanding Track Sizing and Alignment	7
3.1	When Does Track Alignment Work?	7
3.2	Track Sizing Units	7
4	Real-World Examples	8
4.1	Example 1: Centered Card Grid	8
4.2	Example 2: Dashboard Layout	8
4.3	Example 3: Responsive Grid Layout	9
4.4	Example 4: Centered Single Row	10
5	Common Patterns	11
5.1	Pattern 1: Center Everything	11
5.2	Pattern 2: Space Between Columns	11
5.3	Pattern 3: Stretch Rows Evenly	11
5.4	Pattern 4: Top-Left Alignment	12

6 Quick Reference Table	13
6.0.1 Value Quick Reference	13
7 Common Mistakes & Solutions	14
7.1 Mistake 1: Using FR Units with Alignment	14
7.2 Mistake 2: Forgetting Container Size	14
7.3 Mistake 3: Confusing Track Alignment with Item Alignment	15
7.4 Mistake 4: Using space-between When You Want space-evenly	15
8 Browser Support	16
9 Advanced Tips	17
9.1 Tip 1: Combining with Gap	17
9.2 Tip 2: Dynamic Sizing with max-content	17
9.3 Tip 3: Responsive Track Alignment	17
10 Summary: Key Takeaways	19
10.1 Essential Points	19
10.2 Quick Decision Guide	19
10.3 Remember	19
11 Conclusion	20

Chapter 1

Introduction

1.1 What is CSS Grid Tracks Alignment?

Grid tracks are the individual columns and rows in a CSS Grid. Grid tracks alignment allows you to control the space and positioning of these columns and rows within your grid container.

Think of it this way:

- **Grid tracks** = The columns and rows themselves
- **Track alignment** = How those columns and rows are positioned in the available space

Unlike item alignment (which positions content inside cells), track alignment positions the entire tracks (columns or rows) within the container.

1.2 Tracks vs Items: Understanding the Difference

Aspect	Item Alignment	Track Alignment
What it affects	Content inside cells	The columns and rows themselves
Applied to	Individual items or all items	All columns or all rows
Properties	justify-self, align-self	justify-tracks, align-tracks
Properties	justify-items, align-items	justify-content, align-content
Result	Moves content within a cell	Moves entire track in container

1.3 Why Use Track Alignment?

- Create responsive layouts that adapt to container size
- Center a grid inside a large container
- Distribute empty space around columns or rows
- Build professional, balanced designs
- Control spacing without changing grid structure

Chapter 2

Core Track Alignment Properties

2.1 justify-content: Aligning Columns Horizontally

2.1.1 What Does It Do?

The `justify-content` property aligns all grid **columns** horizontally within the container. It only works when the grid is narrower than the container (or smaller).

2.1.2 Available Values

Value	What Happens
<code>start</code>	Columns align to the left
<code>end</code>	Columns align to the right
<code>center</code>	Columns are centered horizontally
<code>stretch</code>	Columns expand to fill container width (default)
<code>space-between</code>	Equal space between columns, none at edges
<code>space-around</code>	Equal space around each column
<code>space-evenly</code>	Equal space everywhere, including edges

2.1.3 Basic Example

```
.grid-container {
  display: grid;
  grid-template-columns: 100px 100px 100px;
  justify-content: center;
  gap: 10px;
  width: 500px; /* Container wider than grid */
  border: 2px solid #999;
  background: #f0f0f0;
}

.item {
  background-color: #3498db;
  padding: 30px;
  color: white;
  text-align: center;
}
```

Result: The three 100px columns are centered in the 500px container. Empty space appears on both sides.

2.1.4 Real Example: space-between

```
.grid-container {
  display: grid;
  grid-template-columns: 100px 100px 100px;
  justify-content: space-between;
  width: 600px;
  border: 2px dashed #ccc;
}

.item {
  background: #e74c3c;
  padding: 20px;
}
```

Result: First column at left edge, last column at right edge, middle column in the center with equal gaps.

2.2 align-content: Aligning Rows Vertically

2.2.1 What Does It Do?

The `align-content` property aligns all grid **rows** vertically within the container. It only works when the grid is shorter than the container.

2.2.2 Available Values

Value	What Happens
<code>start</code>	Rows align to the top
<code>end</code>	Rows align to the bottom
<code>center</code>	Rows are centered vertically
<code>stretch</code>	Rows expand to fill container height (default)
<code>space-between</code>	Equal space between rows, none at edges
<code>space-around</code>	Equal space around each row
<code>space-evenly</code>	Equal space everywhere, including edges

2.2.3 Basic Example

```
.grid-container {
  display: grid;
  grid-template-columns: 100px 100px;
  grid-template-rows: 80px 80px;
  align-content: center;
  gap: 10px;
  height: 400px; /* Container taller than grid */
  border: 2px solid #999;
  background: #f0f0f0;
}
```

```
}

.item {
  background-color: #2ecc71;
  padding: 20px;
  color: white;
  text-align: center;
}
```

Result: The grid (with 2 rows) is centered vertically in the 400px tall container.

2.2.4 Real Example: space-around

```
.grid-container {
  display: grid;
  grid-template-columns: 100px;
  grid-template-rows: 80px 80px 80px;
  align-content: space-around;
  height: 500px;
  border: 2px dashed #ccc;
}

.item {
  background: #9b59b6;
  padding: 20px;
}
```

Result: Three rows with equal space above, between, and below them.

2.3 place-content: Combine Both Alignments

2.3.1 What Does It Do?

The `place-content` shorthand combines `justify-content` and `align-content` in one property.

2.3.2 Syntax

```
place-content: <align-content> <justify-content>;

/* Examples: */
place-content: center center;
place-content: start end;
place-content: space-between space-around;
```

2.3.3 Example

```
.grid-container {
  display: grid;
  grid-template-columns: 100px 100px;
  grid-template-rows: 100px 100px;
```

```
    place-content: center center;
    gap: 15px;
    width: 500px;
    height: 500px;
    border: 2px solid #999;
}

.item {
    background: #f39c12;
    padding: 20px;
}
```

Result: The entire 2x2 grid is centered both horizontally and vertically in the 500x500 container.

Chapter 3

Understanding Track Sizing and Alignment

3.1 When Does Track Alignment Work?

Track alignment only has a visible effect when:

1. Your grid is **smaller than the container**, OR
2. You have **explicit fixed-size tracks** that don't fill the space

Example that DOESN'T show alignment:

```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 1fr; /* fr units stretch */  
  justify-content: center; /* No effect! Columns fill space */  
}
```

Example that DOES show alignment:

```
.grid-container {  
  display: grid;  
  grid-template-columns: 200px 200px; /* Fixed size */  
  justify-content: center; /* Works! Columns don't fill space */  
  width: 600px;  
}
```

3.2 Track Sizing Units

Different sizing units affect how alignment works:

Unit	Description	Alignment Works?
px, em, rem	Fixed size	Yes
1fr, 2fr	Flexible (stretches)	No
auto	Content-based	Depends
min-content	Smallest content fits	Yes
max-content	Largest content fits	Yes
fit-content()	Flexible with limits	Maybe

Chapter 4

Real-World Examples

4.1 Example 1: Centered Card Grid

```
.gallery {  
  display: grid;  
  grid-template-columns: 150px 150px 150px;  
  gap: 20px;  
  place-content: center center;  
  min-height: 600px;  
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
  padding: 20px;  
}  
  
.card {  
  background: white;  
  border-radius: 10px;  
  padding: 30px;  
  box-shadow: 0 4px 15px rgba(0,0,0,0.2);  
  text-align: center;  
}  
  
.card h3 {  
  margin: 0 0 10px 0;  
  color: #333;  
}  
  
.card p {  
  color: #666;  
  font-size: 14px;  
}
```

Use case: Portfolio gallery, product showcase, team members display.

4.2 Example 2: Dashboard Layout

```
.dashboard {  
  display: grid;
```

```
    grid-template-columns: 250px 250px;
    grid-template-rows: 150px 150px;
    justify-content: space-around;
    align-content: space-evenly;
    gap: 20px;
    min-height: 700px;
    background: #f5f7fa;
    padding: 30px;
}

.widget {
    background: white;
    border-radius: 8px;
    padding: 25px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.08);
    border-left: 4px solid #3498db;
}

.widget.success {
    border-left-color: #2ecc71;
}

.widget.warning {
    border-left-color: #f39c12;
}
```

Use case: Admin panels, analytics dashboards, monitoring screens.

4.3 Example 3: Responsive Grid Layout

```
.responsive-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
    gap: 15px;
    place-content: center start;
    padding: 20px;
}

@media (max-width: 600px) {
    .responsive-grid {
        grid-template-columns: repeat(auto-fit, minmax(120px, 1fr));
        place-content: start start;
    }
}

.item {
    background: #e74c3c;
    padding: 20px;
    border-radius: 5px;
    text-align: center;
    color: white;
}
```

Use case: Responsive product grids, flexible layouts that adapt to screen size.

4.4 Example 4: Centered Single Row

```
.button-row {
  display: grid;
  grid-template-columns: 120px 120px 120px;
  gap: 10px;
  justify-content: center;
  align-content: center;
  height: 200px;
  background: #ecf0f1;
  border-radius: 8px;
}

.btn {
  background: #3498db;
  color: white;
  border: none;
  padding: 15px 25px;
  border-radius: 5px;
  cursor: pointer;
  font-weight: bold;
}

.btn:hover {
  background: #2980b9;
}
```

Use case: Button groups, action toolbars, control panels.

Chapter 5

Common Patterns

5.1 Pattern 1: Center Everything

Center the grid both horizontally and vertically:

```
.container {  
  display: grid;  
  grid-template-columns: 200px 200px;  
  grid-template-rows: 100px 100px;  
  place-content: center;  
  width: 600px;  
  height: 500px;  
}
```

5.2 Pattern 2: Space Between Columns

Distribute columns evenly with edges at container edges:

```
.container {  
  display: grid;  
  grid-template-columns: 100px 100px 100px;  
  justify-content: space-between;  
  width: 500px;  
}
```

5.3 Pattern 3: Stretch Rows Evenly

Distribute rows with equal spacing:

```
.container {  
  display: grid;  
  grid-template-rows: 80px 80px;  
  align-content: space-evenly;  
  height: 400px;  
}
```

5.4 Pattern 4: Top-Left Alignment

Push all tracks to the top-left corner:

```
.container {  
  display: grid;  
  grid-template-columns: 150px 150px;  
  grid-template-rows: 100px 100px;  
  place-content: start start;  
  width: 600px;  
  height: 500px;  
}
```

Chapter 6

Quick Reference Table

Property	Affects	Direction	What It Does
<code>justify-content</code>	Columns	Horizontal	Aligns all columns left/right/center
<code>align-content</code>	Rows	Vertical	Aligns all rows top/middle/bottom
<code>place-content</code>	Both	Both	Shorthand for align + justify

6.0.1 Value Quick Reference

Value	Result
<code>start</code>	Align to beginning (left or top)
<code>end</code>	Align to end (right or bottom)
<code>center</code>	Align in middle
<code>stretch</code>	Expand to fill container
<code>space-between</code>	Equal space between, nothing at edges
<code>space-around</code>	Equal space around each track
<code>space-evenly</code>	Equal space everywhere including edges

Chapter 7

Common Mistakes & Solutions

7.1 Mistake 1: Using FR Units with Alignment

Wrong:

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  justify-content: center; /* No effect! */  
}
```

Why it doesn't work: fr (fractional) units stretch to fill space, so there's no room for alignment.

Correct:

```
.container {  
  display: grid;  
  grid-template-columns: 150px 150px 150px;  
  justify-content: center;  
  width: 600px; /* Container larger than grid */  
}
```

7.2 Mistake 2: Forgetting Container Size

Problem: Track alignment doesn't work if the container is the same size as the grid.

Wrong:

```
.container {  
  display: grid;  
  grid-template-columns: 200px;  
  justify-content: center;  
  /* No width specified - container is 200px */  
}
```

Correct:

```
.container {  
  display: grid;
```

```
grid-template-columns: 200px;
justify-content: center;
width: 500px; /* Container larger than grid */
}
```

7.3 Mistake 3: Confusing Track Alignment with Item Alignment

Wrong:

```
.item {
  justify-content: center; /* Items don't have content to justify */
}
```

Correct:

```
.container {
  justify-content: center; /* Aligns tracks */
}

.item {
  display: flex;
  justify-content: center; /* Aligns content inside item */
}
```

7.4 Mistake 4: Using space-between When You Want space-evenly

Problem: space-between leaves no space at the edges.

```
/* space-between: [gap] col1 [gap] col2 [gap] col3 [gap] */
justify-content: space-between;

/* space-evenly: [gap] col1 [gap] col2 [gap] col3 [gap] */
justify-content: space-evenly;

/* space-around: [gap/2] col1 [gap] col2 [gap] col3 [gap/2] */
justify-content: space-around;
```

Chapter 8

Browser Support

All modern browsers fully support CSS Grid track alignment:

Property	Chrome	Firefox	Safari	Edge
<code>justify-content</code>	57+	52+	10.1+	16+
<code>align-content</code>	57+	52+	10.1+	16+
<code>place-content</code>	59+	53+	10.1+	79+

All modern browsers released in the last 5 years support these properties.

Chapter 9

Advanced Tips

9.1 Tip 1: Combining with Gap

```
.container {  
  display: grid;  
  grid-template-columns: 100px 100px;  
  gap: 20px;  
  justify-content: space-between;  
  width: 500px;  
}  
  
/* Gap adds space between items, justify-content adds space around */
```

9.2 Tip 2: Dynamic Sizing with max-content

```
.container {  
  display: grid;  
  grid-template-columns: max-content max-content max-content;  
  justify-content: center;  
  gap: 15px;  
}  
  
/* Columns size to content, then center the group */
```

9.3 Tip 3: Responsive Track Alignment

```
.container {  
  display: grid;  
  grid-template-columns: 150px 150px;  
  place-content: center;  
}  
  
@media (max-width: 768px) {  
  .container {
```

```
        grid-template-columns: 120px;  
        place-content: start;  
    }  
}
```

Chapter 10

Summary: Key Takeaways

10.1 Essential Points

- **Track alignment** = Positioning columns and rows in the container
- **justify-content** = Aligns columns horizontally
- **align-content** = Aligns rows vertically
- **place-content** = Shorthand for both
- **Only works** when grid is smaller than container
- **Don't use** with **fr** units (they auto-stretch)

10.2 Quick Decision Guide

1. Want to center columns? → Use `justify-content: center`
2. Want to center rows? → Use `align-content: center`
3. Want to center everything? → Use `place-content: center`
4. Want space between columns? → Use `justify-content: space-between`
5. Want equal space everywhere? → Use `justify-content: space-evenly`

10.3 Remember

Track alignment is about **positioning the grid structure itself**, not the content inside the cells. It works best with:

- Fixed-size tracks (`px`, `em`, `rem`)
- Containers larger than the grid
- Modern, clean layouts

Chapter 11

Conclusion

CSS Grid track alignment gives you powerful control over how your grid behaves in its container. By mastering `justify-content`, `align-content`, and `place-content`, you can create flexible, professional layouts that adapt to any design requirement.

Start with `place-content: center` for quick centering, then explore spacing options as your designs become more complex.

Happy grid building!